

Practical - 5

WEB DEVELOPMENT FRAMEWORKS (ITUE203)

SEMESTER: 3RD, 2026-27

ID: 25DCE110

Practical Title: Registration Form with Frontend Validation and User-Friendly Error Handling

Problem Definition : Create a student registration form with HTML5 input types and JavaScript validation for name, email, mobile number, password, confirm password, course, year, gender, and terms acceptance. Use Regular Expression for validations.

A. Theory

1. Are correct input types and attributes used?

Mostly yes. You're using `type="email"` for the email field and `type="tel"` for mobile, which tells the browser (and mobile keyboards) what kind of input to expect — on a phone, `type="tel"` will pop up a numeric keypad instead of a full keyboard, which is a nice touch. The two password fields correctly use `type="password"` so the text gets masked. One gap: you're not using any native HTML5 attributes like `required`, `maxlength`, or `pattern` on the inputs — right now all validation happens only in JavaScript. This works, but if JavaScript fails to load for some reason, the form has zero validation left. Adding `required` and a rough pattern as a backup is good practice, even though your JS is the real gatekeeper.

2. Are validation errors displayed near the relevant fields?

Yes, and this is done well. Every input has its own `<span id="...Error"></span>` placed right after it in the HTML, and your JS writes the specific error message into that exact span. So if the mobile number is wrong, the error shows up right under the mobile field, not in some generic message box at the top. This is good UX because the user doesn't have to hunt for what went wrong.

3. Is password strength checked?

Yes. There's a keyup listener on the password field that checks the value on every keystroke and updates a `<p id="strength">` element live, showing "Weak," "Medium," or "Strong" based on length and whether it has uppercase, lowercase, numbers, and special characters. This gives the user real-time feedback as they type, rather than only finding out on submit.

4. Is the form accessible using keyboard and screen-reader-friendly labels?

Partly. Every input has a proper `<label for="...">` matching the input's id (name, email, mobile, password, course, year, gender radios, terms checkbox) — this is the single most important thing for screen readers, since it lets them announce "Name, edit text" instead of just "edit text." Keyboard-wise, you're using standard `<input>`, `<select>`, radio, and checkbox elements, which are all natively focusable and operable with Tab/Space/Arrow keys by default — you haven't broken that with any custom widgets, so that part is solid.

The gap is that your error `<span>` elements have no `aria-live` attribute. That means when JavaScript injects an error message into a span, a screen reader user won't hear it — visually the text appears, but nothing announces it out loud since the page doesn't "know" to speak on that change. Adding `aria-live="polite"` to each error span (or one shared live region) would fix that. It'd also help to add `aria-required="true"` on required fields for extra clarity to assistive tech.

B. Implementation/ Output/Result

1. Element references

Used: `document.getElementById()`

Implementation: Grabbed each input, span, and the form once at the start, instead of searching the DOM repeatedly

2. Name validation

Used: Regex `/^[A-Za-z ]{3,}$/`

Implementation: Tested with `.test()` — only letters and spaces allowed, minimum 3 characters

3. Enrollment validation

Used: Regex `/^D?\d{2}DCE\d{3}$/`

Implementation: Optional leading D, 2-digit year, literal DCE, 3-digit roll number

4. Email validation

Used: Regex `/^[a-zA-Z0-9]+@charusat.edu.in$/`

Implementation: Alphanumeric local part, must end exactly with `@charusat.edu.in`

5. Mobile validation

Used: Regex `/^[6-9]\d{9}$/`

Implementation: Starts with 6-9, followed by exactly 9 more digits (10 digits total)

6. Password validation

Used: Regex `/^(?=.*[a-z])(?=.*[A-Z])(?=.*[0-9])(?=.*[@$!%*?&]).{8,}$/`
 Implementation: Lookaheads confirm at least one lowercase, uppercase, number, and special character; minimum 8 characters total

HTML webpages screenshots:

Registration page:

The screenshot shows a 'Student Registration' form with the following fields and labels:

- Name:** Enter your full name
- Enrollment ID:** Enter your enrollment id
- Email:** Enter your email
- Mobile Number:** Enter 10 digit mobile number
- Password:** Enter password
- Confirm Password:** Re-enter password
- Course:** Select Course
- Year:** Select Year
- Gender:** Male, Female, Other
- ☐ I agree to the terms and conditions
- Buttons:** Register, Clear
- Footer:** Home, FAQ, Feedback, About

Invalid Enrollment Id:

The screenshot shows an 'Enrollment ID' input field with the value '25000'. Below the field, a red error message reads: "Enrollment ID must be in the format 25DCE001 or D26DCE126".

Invalid Email:

The screenshot shows an 'Email' input field with the value '25dce110@gmail'. Below the field, a red error message reads: "Enter a valid university email (e.g. name@charusat.edu.in)".

Invalid Mobile number:

The screenshot shows a 'Mobile Number' input field with the value '989898'. Below the field, a red error message reads: "Mobile number must be exactly 10 digits and start with 6-9".

Invalid Password:

25DCE110 – 3CE2

Password:

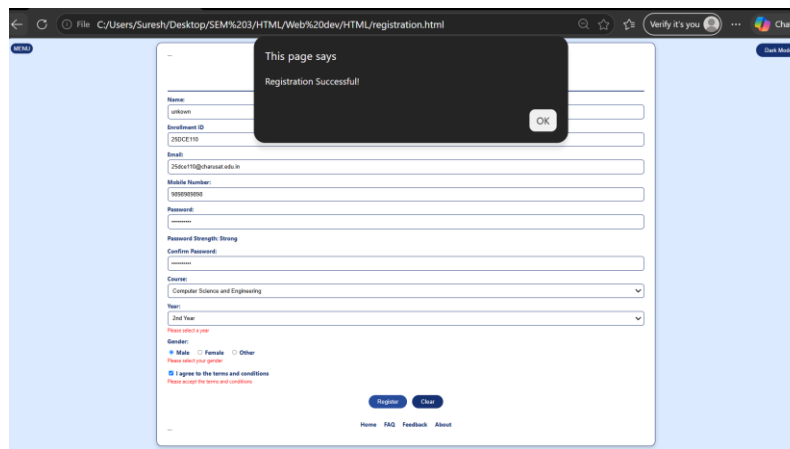
Password must be at least 8 characters and include uppercase, lowercase, number and special character

Password must be same:

Confirm Password:

Passwords do not match

Valid Details:



The screenshot shows a web browser window with the address bar displaying 'C:/Users/Suresh/Desktop/SEM%203/HTML/Web%20dev/HTML/registration.html'. A dark overlay box in the center of the page says 'This page says Registration Successful!' with an 'OK' button. The registration form is visible in the background, containing fields for Name, Enrollment ID, Email, Mobile Number, Password, Confirm Password, Course (dropdown), Year (dropdown), Gender (radio buttons), and a checkbox for 'I agree to the terms and conditions'. At the bottom of the form are 'Register' and 'Clear' buttons. The footer of the page includes links for 'Home', 'FAQ', 'Feedback', and 'About'.

Description:

In this practical the main focus is on making a student registration form. And front end validating and user friendly error handling using JAVASCRIPT. The most important part of Javascript , REGEX is used to define the format of input fields to validate the user data.

GitHub Repository Link:

<https://github.com/kanan2007s-sys/Web-development-/tree/main>

Challenges Faced:

I faced many challenges in making the functions in Javascript and other challenge was to create a valid regex in code. Else understanding the task was not hard but implementing was.

Learning Outcomes:

I learned how to use Regex in webpages using Javascript.

Faculty Remarks: